Phantom

PHANTOM

Autonomous Adversary Simulation Platform

AI-Powered Penetration Testing Agent

Comprehensive System Report

Version 0.9.20 — March 2026

Author: Gadouri Rodwan

Supervisor: Dr. Allama Oussama

Institution: ESTIN — École Supérieure en Sciences et Technologies de l'Informatique et du Numérique

Date: March 2, 2026

808+ Tests • **83-Finding Audit** • **30+ Security Tools** • **7-Layer Defense** • **Multi-Agent AI**

Contents

Abstract **iv**

1 Introduction **1**
1.1 Motivation and Context . . . 1
1.2 Project Objectives . . . 1
1.3 Document Structure . . . 2

2 System Architecture **3**
2.1 Architectural Overview . . . 3
2.2 Layer Responsibilities . . . 3
2.2.1 Interface Layer . . . 4
2.2.2 Orchestration Layer . . . 4
2.2.3 Agent Core . . . 4
2.2.4 Security Layer . . . 4
2.2.5 Execution Layer . . . 5
2.2.6 Output Layer . . . 5
2.3 Component Interaction . . . 5

3 Agent Core: The ReAct Reasoning Engine **7**
3.1 The ReAct Pattern . . . 7
3.2 Agent State Management . . . 8
3.3 Multi-Agent Delegation . . . 8
3.4 Memory Compression . . . 9

4 LLM Integration **10**
4.1 Provider Abstraction . . . 10
4.2 Cost Control . . . 10
4.3 Prompt Engineering . . . 11

5 Security Architecture **12**
5.1 7-Layer Defense Model . . . 12
5.1.1 Layer 1: Scope Validator . . . 12
5.1.2 Layer 2: Tool Firewall . . . 13
5.1.3 Layer 3: Docker Sandbox . . . 13
5.1.4 Layer 4: Cost Controller . . . 13
5.1.5 Layer 5: Time Limits . . . 13
5.1.6 Layer 6: HMAC Audit Trail . . . 13
5.1.7 Layer 7: Output Sanitizer . . . 14
5.2 Security Audit Results . . . 14

6 Execution Runtime **15**
6.1 Docker Sandbox Design . . . 15
6.2 Tool Server Protocol . . . 15
6.3 Container Lifecycle . . . 16

7 Tool Ecosystem **17**
7.1 Tool Categories . . . 17
7.2 Tool Integration Pattern . . . 17
7.3 Tool Selection Intelligence . . . 18

8 Report Pipeline **19**
8.1 7-Stage Enrichment Pipeline . . . 19
8.2 Output Formats . . . 20
8.3 Compliance Mapping . . . 20

9 Knowledge & Memory System **21**
9.1 Persistent Knowledge Store . . . 21
9.2 Attack Graph . . . 21
9.3 Scan Resume (Checkpoint System) . . . 21

10 Data Models **23**
10.1 Vulnerability Model . . . 23
10.2 Scan Configuration Model . . . 23

11 Scan Lifecycle **24**
11.1 End-to-End Flow . . . 24
11.2 Scan Profiles . . . 25

12 Testing & Quality Assurance **26**
12.1 Test Strategy . . . 26
12.2 Test Results . . . 26
12.3 CI/CD Pipeline . . . 26

13 Live Validation: OWASP Juice Shop **27**
13.1 Test Environment . . . 27
13.2 Scan Results . . . 27
13.3 Tool Execution Statistics . . . 28
13.4 Compliance Mapping Results . . . 28
13.5 Resume Feature Validation . . . 28
13.6 Generated Artifacts . . . 29

14 Future Work **30**
14.1 Short-Term Roadmap (v1.0) . . . 30
14.2 Medium-Term Research . . . 30
14.3 Long-Term Vision . . . 30

15 Conclusion **32**

References **33**

List of Figures

2.1 Six-layer system architecture of 3
2.2 Component interaction during a scan. 6

3.1 The ReAct reasoning cycle. 7
3.2 Multi-agent delegation hierarchy. 8

5.1 The 7-layer security defense model. 12

8.1 The 7-stage post-scan enrichment pipeline. 19

11.1 Six-phase scan lifecycle. 24

List of Tables

3.1 Agent state fields. 8

4.1 Supported LLM providers. 10

5.1 Security audit findings summary. 14

6.1 Sandbox container specifications. 15

7.1 Security tool inventory by category. 17

8.1 Report output formats. 20

11.1 Available scan profiles. 25

12.1 Test suite breakdown. 26

13.1 Live scan configuration. 27
13.2 Tool usage during the OWASP Juice Shop scan. 28
13.3 Compliance violations identified. 28
13.4 Output files from the OWASP Juice Shop scan. 29

Abstract

This report presents , an autonomous AI-powered penetration testing platform that leverages large language models (LLMs) to conduct end-to-end security assessments with minimal human intervention. Unlike traditional vulnerability scanners that rely on static signatures and predefined rule sets, employs a Reason-then-Act (ReAct) cognitive loop that enables it to analyze target environments, formulate attack strategies, chain multi-step exploits, and adapt its approach based on observed behavior.

The system integrates over 30 industry-standard security tools—including nmap, nuclei, sqlmap, ffuf, httpx, and katana—executing all operations within an ephemeral Docker sandbox that provides strict isolation from the host system. A multi-agent architecture enables parallel reconnaissance, exploitation, and verification tasks, while a 7-layer security framework ensures that the platform itself cannot be weaponized beyond its intended scope.

Key contributions of this work include:

- An autonomous agent architecture achieving human-comparable penetration testing without manual intervention
- A sandboxed execution environment with defense-in-depth security controls
- Automated vulnerability verification producing working proof-of-concept exploits
- MITRE ATT&CK enrichment and multi-framework compliance mapping (OWASP, PCI DSS, NIST)
- A cross-scan knowledge persistence system enabling progressive intelligence
- Comprehensive testing (808+ tests, 0 failures) and a fully resolved 83-finding security audit

Keywords: penetration testing, autonomous agents, large language models, cybersecurity, vulnerability assessment, Docker sandboxing, ReAct reasoning

Chapter 1

Introduction

1.1 Motivation and Context

The cybersecurity landscape has undergone a fundamental transformation. Attack surfaces have expanded dramatically with cloud-native architectures, microservices, APIs, and single-page applications. Meanwhile, the global shortage of skilled penetration testers—estimated at 3.4 million professionals by 2025 (ISC^2 Cybersecurity Workforce Study)—leaves organizations unable to assess their security posture with adequate frequency or depth.

Traditional vulnerability scanners, while useful for baseline checks, suffer from well-documented limitations:

- **Signature dependence:** They can only detect vulnerabilities matching predefined patterns, missing novel or chained attack vectors.
- **High false positive rates:** Studies consistently show 40–70% false positive rates, consuming valuable analyst time.
- **Shallow coverage:** Single-pass scanning cannot discover multi-step exploits that require sequential reasoning.
- **No contextual reasoning:** They lack the ability to interpret application behavior and adapt strategies accordingly.

These limitations create a critical gap: organizations need human-level penetration testing, but cannot scale human expertise to match the speed and breadth of modern software deployment.

1.2 Project Objectives

was designed to bridge this gap by creating an autonomous AI agent that conducts penetration tests with the reasoning capabilities of a skilled human tester. The specific objectives are:

1. **Autonomous operation:** Conduct end-to-end security assessments—from reconnaissance through exploitation to reporting—without human intervention.

2. **Adaptive reasoning:** Use LLM-powered reasoning to analyze targets, plan attack strategies, and adapt based on observed results.
3. **Verified findings:** Produce working proof-of-concept exploits for every confirmed vulnerability, eliminating false positives.
4. **Security-first design:** Ensure the platform itself is secure through defense-in-depth, sandboxing, and strict scope enforcement.
5. **Actionable reporting:** Generate MITRE ATT&CK-enriched, compliance-mapped reports with concrete remediation guidance.

1.3 Document Structure

This report is organized as follows:

Chapter 2 — System Architecture
High-level design, layer decomposition, and data flow.

Chapter 3 — Agent Core
The ReAct reasoning loop, state management, and multi-agent delegation.

Chapter 4 — LLM Integration
Model abstraction, memory compression, and cost control.

Chapter 5 — Security Architecture
The 7-layer defense model protecting the platform.

Chapter 6 — Execution Runtime
Docker sandbox design, tool server protocol, and isolation.

Chapter 7 — Tool Ecosystem
The 30+ security tools and their integration patterns.

Chapter 8 — Report Pipeline
Vulnerability enrichment, compliance mapping, and output formats.

Chapter 9 — Knowledge System
Cross-scan persistence, false positive learning, and attack graphs.

Chapter 10 — Data Models
Pydantic schemas and domain object design.

Chapter 11 — Scan Lifecycle
End-to-end walkthrough from initialization to final report.

Chapter 12 — Testing & Quality Assurance
Test strategy, coverage, and security audit results.

Chapter 13 — Live Validation
Real-world scan against OWASP Juice Shop.

Chapter 14 — Future Work
Roadmap and research directions.

Chapter 2

System Architecture

2.1 Architectural Overview

follows a layered architecture designed for separation of concerns, security isolation, and extensibility. The system comprises six primary layers, each with well-defined responsibilities and interfaces.

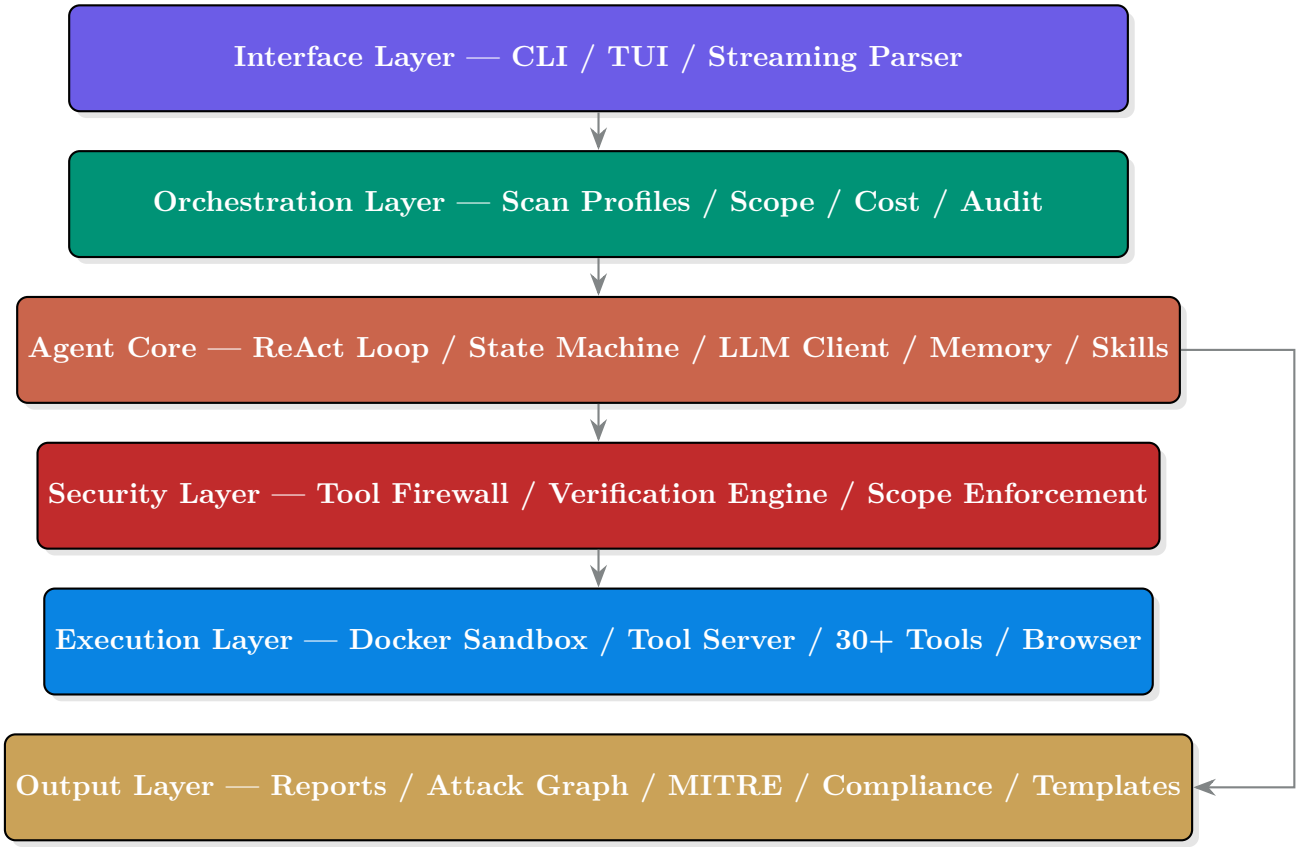

Figure 2.1: Six-layer system architecture of .

2.2 Layer Responsibilities

2.2.1 Interface Layer

The interface layer provides three interaction modes:

- **CLI (Command Line Interface):** Standard command-line invocation with flags for target, scan mode, profile, and instructions.
- **TUI (Terminal User Interface):** Rich interactive mode built with Textual, providing real-time scan visualization, agent tree, and live findings.
- **Streaming Parser:** Processes LLM responses token-by-token, extracting tool calls from partial XML streams without waiting for complete responses.

2.2.2 Orchestration Layer

Manages the lifecycle of a scan:

- **Scan Profiles:** Predefined configurations (quick, standard, deep, stealth, api_only) controlling iteration limits, tool selection, and timeout thresholds.
- **Scope Validator:** Ensures all tool executions target only authorized hosts and ports.
- **Cost Controller:** Tracks per-request and cumulative LLM costs with configurable budget ceilings.
- **Audit Logger:** HMAC-signed, append-only event log providing tamper-evident forensic trail.

2.2.3 Agent Core

The cognitive engine of (detailed in Chapter 3):

- **BaseAgent (ReAct Loop):** The primary reasoning cycle: Observe → Reason → Plan → Act → Record.
- **State Machine:** Bounded agent state with memory limits, preventing context overflow.
- **LLM Client:** Abstraction over LiteLLM supporting 100+ providers with automatic retry, fallback, and key rotation.
- **Memory Compressor:** Progressively summarizes older conversation turns to stay within context limits.
- **Skills Engine:** Injects domain-specific knowledge (OWASP Top 10, common CVEs, tool usage guides) into agent prompts.

2.2.4 Security Layer

A defense-in-depth barrier between reasoning and execution (detailed in Chapter 5):

- **Tool Firewall:** Validates every tool call against allowlists, argument constraints, and scope rules before execution.
- **Verification Engine:** Cross-validates findings through independent re-exploitation.

- **Scope Enforcement:** Runtime checks ensuring tools cannot reach unauthorized targets.

2.2.5 Execution Layer

All tool execution occurs inside Docker containers (detailed in Chapter 6):

- **Docker Sandbox:** Ephemeral Kali-based container (14 GB) with restricted capabilities.
- **Tool Server:** HTTP API running inside the container, authenticated with per-scan tokens.
- **30+ Security Tools:** nmap, nuclei, sqlmap, ffuf, httpx, katana, semgrep, nikto, gobuster, arjun, and more.
- **Playwright Browser:** Headless Chromium for JavaScript-heavy application testing.

2.2.6 Output Layer

Post-scan enrichment and reporting (detailed in Chapter 8):

- **Reports:** Structured JSON, styled HTML, and Markdown output.
- **Attack Graph:** NetworkX-based directed graph modeling attack paths.
- **MITRE ATT&CK:** Automatic CWE, CAPEC, and technique-level enrichment.
- **Compliance Mapper:** OWASP Top 10, PCI DSS v4, and NIST 800-53 control mapping.
- **Nuclei Templates:** Auto-generated YAML templates for reproducible scanning.

2.3 Component Interaction

Figure 2.2 shows the primary data flow during a scan.

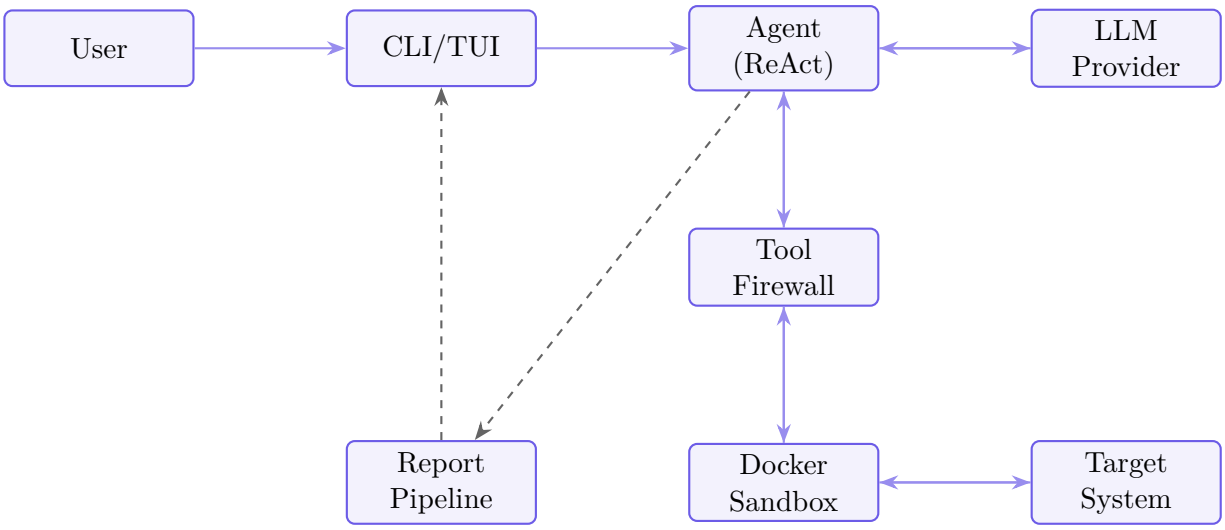

Figure 2.2: Component interaction during a scan.

Chapter 3

Agent Core: The ReAct Reasoning Engine

3.1 The ReAct Pattern

uses the **ReAct** (Reasoning + Acting) paradigm, where the agent alternates between thinking about observations and taking actions. This contrasts with chain-of-thought prompting (reasoning only) or simple tool-calling (acting without explicit reasoning).

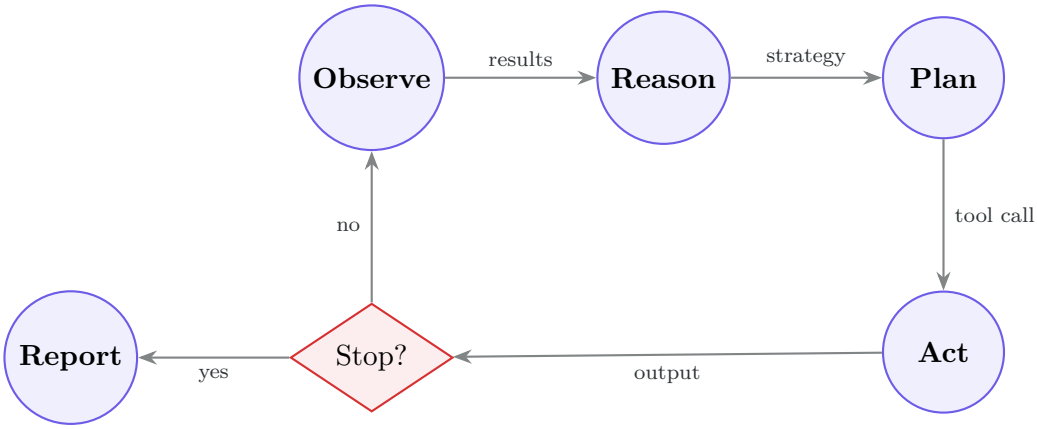

Figure 3.1: The ReAct reasoning cycle.

Each iteration of the loop:

1. **Observe:** The agent receives tool output from the previous action (or the initial scan context).
2. **Reason:** The LLM analyzes the current state, identifies patterns, and determines what information is still needed.
3. **Plan:** The agent selects a specific tool and constructs its arguments.
4. **Act:** The tool call is validated by the firewall and executed in the sandbox.
5. **Record:** Results are stored in the agent state and findings ledger.

6. **Check Stop:** If the iteration budget is exhausted, time limit reached, or findings are complete, the agent finalizes and generates reports.

3.2 Agent State Management

Each agent maintains a bounded state object (`AgentState`) with the following properties:

Table 3.1: Agent state fields.

Field	Description
`agent_id`	UUID identifying this agent instance
`agent_name`	Human-readable name (e.g., "Recon Agent")
`iteration`	Current iteration count
`max_iterations`	Budget limit from scan profile
`task`	The natural-language task description
`parent_id`	Parent agent UUID (null for root)
`conversation`	Bounded message history with compression
`start_time`	UTC timestamp of agent creation

The `EnhancedAgentState` extends this with vulnerability tracking, endpoint coverage, host discovery, and scan phase management.

3.3 Multi-Agent Delegation

The root agent can create specialized sub-agents for parallel or focused work:

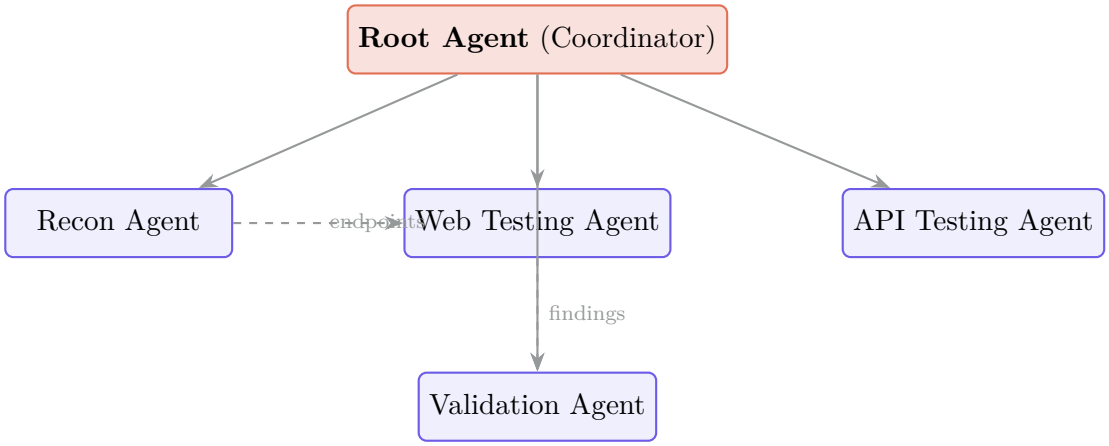

Figure 3.2: Multi-agent delegation hierarchy.

Each sub-agent:

- Receives a focused task description from the parent.
- Inherits the sandbox session (same Docker container).
- Has its own independent state, conversation history, and iteration budget.

- Returns a structured result summary to the parent upon completion.

3.4 Memory Compression

As the conversation history grows, applies progressive **memory compression**:

1. **Threshold Detection:** When the conversation exceeds the memory threshold (configurable per profile), compression is triggered.
2. **Summarization:** Older turns are condensed into a brief summary preserving key findings, discovered endpoints, and tool results.
3. **Retention:** The most recent k turns are kept verbatim to maintain immediate context.
4. **Cost Savings:** Compression reduces token usage by 40–60% on long scans.

Chapter 4

LLM Integration

4.1 Provider Abstraction

uses LiteLLM to abstract over 100+ LLM providers through a unified interface. This enables:

Table 4.1: Supported LLM providers.

Provider	Model Example	Free Tier
OpenAI	`openai/gpt-4o`	✗
Anthropic	`anthropic/claude-sonnet-4-20250514`	✗
Google	`gemini/gemini-2.5-pro`	✓
Groq	`groq/llama-3.3-70b-versatile`	✓
DeepSeek	`deepseek/deepseek-chat`	✗
OpenRouter	`openrouter/deepseek/deepseek-v3.2`	✗
Ollama	`ollama/llama3.1`	✓
Azure	`azure/gpt-4o`	✗

4.2 Cost Control

Cost management is critical for production use:

Cost Control Mechanisms

- **Per-Request Ceiling:** Maximum cost per individual LLM call (default: $5.00).
- **Scan Budget:** Total maximum cost per scan (default: $25.00).
- **Token Tracking:** Input, output, and cached tokens tracked per request.
- **Key Rotation:** Comma-separated API keys with automatic rotation on rate limits.
- **Cost Estimation:** Pre-request cost estimation to avoid budget overruns.

4.3 Prompt Engineering

The prompt system uses Jinja2 templates organized by agent type:

- **System Prompt:** Defines the agent's identity, capabilities, security rules, and output format.
- **Skills Injection:** Dynamically loads relevant domain knowledge (OWASP, CVE databases, tool guides) based on scan context.
- **State Serialization:** Current findings, endpoints, and tool history injected into each turn.
- **Anti-Injection Guards:** HTML tag stripping, length caps, and meta-instruction filtering on user-supplied instructions.

Chapter 5

Security Architecture

5.1 7-Layer Defense Model

implements defense-in-depth through seven security layers:

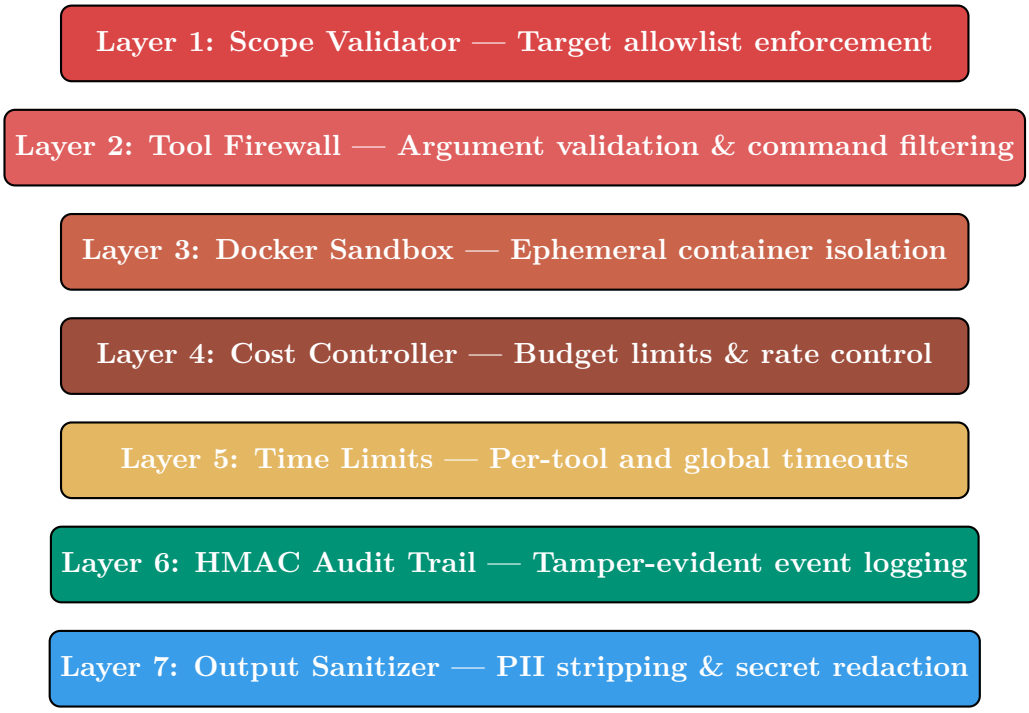

Figure 5.1: The 7-layer security defense model.

5.1.1 Layer 1: Scope Validator

Before any tool execution, the Scope Validator ensures the target is within the authorized scope:

- Parses the scan target into hostname, IP, port, and path components.
- All tool calls are checked against the scope allowlist.

- SSRF protection: blocks tool calls to internal networks, cloud metadata, and link-local addresses.
- Configurable explicit deny lists for sensitive ranges.

5.1.2 Layer 2: Tool Firewall

The Tool Firewall inspects every tool invocation before it reaches the sandbox:

- **Allowlist:** Only pre-registered tools can be invoked.
- **Argument Validation:** Parameters checked against expected types, lengths, and patterns.
- **Command Injection Prevention:** Shell metacharacters stripped from tool arguments.
- **Rate Limiting:** Prevents excessive calls to any single tool.

5.1.3 Layer 3: Docker Sandbox

All offensive operations execute inside ephemeral Docker containers:

- **No host filesystem access:** Container has no volume mounts to the host.
- **Restricted capabilities:** Only `NET_ADMIN` and `NET_RAW` (needed for nmap).
- **Automatic cleanup:** Container destroyed after scan completion.
- **Token-authenticated API:** Tool server requires a per-scan secret token.
- **Network isolation:** Container can only reach the target and host gateway.

5.1.4 Layer 4: Cost Controller

Prevents runaway spending:

- Per-request cost ceiling (default: $5.00 per LLM call).
- Global scan budget (default: $25.00).
- Graceful degradation: agent completes current phase and generates partial report when budget exceeded.

5.1.5 Layer 5: Time Limits

- **Per-tool timeout:** Configurable (default: 600s per sandbox execution).
- **Global scan timeout:** Enforced at the orchestration layer.
- **LLM timeout:** Individual request timeouts with retry logic.

5.1.6 Layer 6: HMAC Audit Trail

- Every event (tool call, finding, agent creation) written to an append-only JSONL file.

- Each entry includes an HMAC signature computed with a per-scan key.
- Chain integrity: each entry references the hash of the previous entry.
- Enables post-hoc forensic analysis and tamper detection.

5.1.7 Layer 7: Output Sanitizer

- Strips PII patterns (emails, SSNs, credit card numbers) from reports.
- Redacts API keys, tokens, and credentials discovered during scanning.
- Ensures report content is safe for distribution.

5.2 Security Audit Results

Two comprehensive offensive audits were performed on the codebase:

Table 5.1: Security audit findings summary.

Severity	Found	Fixed	Verified	Remaining
Critical	8	8	8	0
High	19	19	19	0
Medium	34	34	34	0
Low	22	22	22	0
Total	**83**	**83**	**83**	**0**

Audit Outcome

All 83 findings (8 Critical, 19 High, 34 Medium, 22 Low) were resolved and verified with dedicated regression tests. Final system score: **8.0/10**.

Chapter 6

Execution Runtime

6.1 Docker Sandbox Design

The sandbox is a pre-built Docker image based on Kali Linux, approximately 14 GB in size, containing all security tools and runtime dependencies.

Table 6.1: Sandbox container specifications.

Property	Value
Base image	Kali Linux (custom Phantom build)
Image size	∼14 GB
Tool count	30+ offensive security tools
Runtime	Python 3.12, Bash, Playwright + Chromium
Proxy	Caido intercept proxy
Tool Server port	48081 (token-authenticated HTTP API)
Capabilities	`NET_ADMIN`, `NET_RAW` only
Lifecycle	Created per-scan, destroyed on completion

6.2 Tool Server Protocol

The Tool Server is an HTTP API running inside the container that mediates all tool executions:

Listing 6.1: Simplified tool server request flow.

```
# Host side (Phantom CLI)
POST http://127.0.0.1:{port}/execute
Authorization: Bearer {scan_token}
Content-Type: application/json

{
    "tool": "nmap_scan",
    "args": {"target": "10.0.0.1", "flags": "-sV -sC"},
    "timeout": 300
}

```

```
# Response
{
    "status": "success",
    "output": "PORT   STATE SERVICE VERSION\n22/tcp open ssh ...",
    "duration_ms": 12340,
    "truncated": false
}
```

Tool output is automatically truncated to 8 KB to prevent context overflow in the LLM conversation.

6.3 Container Lifecycle

1. **Creation:** Container started with `sleep infinity`, port mapping, and environment variables.
2. **Initialization:** Entrypoint script starts Caido proxy, configures SSL certificates, and launches the tool server.
3. **Health Check:** Phantom polls `/health` until the tool server reports ready.
4. **Execution:** Tool calls dispatched via HTTP API throughout the scan.
5. **Cleanup:** Container stopped and removed; no persistent state remains.

Chapter 7

Tool Ecosystem

7.1 Tool Categories

integrates 30+ security tools across five categories:

Table 7.1: Security tool inventory by category.

Category	Tools	Purpose
Reconnaissance	nmap, httpx, katana, whatweb	Port scanning, technology fingerprinting, endpoint discovery
Fuzzing	ffuf, gobuster, arjun, wfuzz	Directory brute-forcing, parameter discovery
Vulnerability	nuclei, nikto, semgrep	Known CVE detection, misconfigurations, code analysis
Exploitation	sqlmap, commix, XSStrike	SQL injection, command injection, XSS exploitation
Browser	Playwright (Chromium)	JavaScript rendering, form interaction, SPA testing
Utility	curl, python_action, terminal_execute	Custom scripting, API interaction, arbitrary commands

7.2 Tool Integration Pattern

Each tool follows a consistent integration pattern:

1. **Schema Definition:** JSON schema defining the tool's name, description, and parameters.

2. **Firewall Registration:** Tool registered with allowed argument patterns and scope rules.
3. **Execution Wrapper:** Python function that translates schema parameters to CLI commands.
4. **Output Parsing:** Structured parsing of tool output (JSON, XML, or text) into findings.
5. **Result Truncation:** Output capped at 8 KB with intelligent summarization.

7.3 Tool Selection Intelligence

The agent's tool selection is guided by:

- **Scan Profile:** Priority tools and skip lists defined per profile.
- **Skills Knowledge:** Domain expertise files recommend tool chains for specific vulnerability classes.
- **Context Learning:** Results from previous tools inform the next selection (e.g., finding an API endpoint triggers sqlmap).
- **Efficiency Rules:** The system enforces rules like "prefer `python_action` for batch requests over individual `send_request` calls."

Chapter 8

Report Pipeline

8.1 7-Stage Enrichment Pipeline

Every scan runs a post-scan enrichment pipeline that transforms raw findings into actionable intelligence:

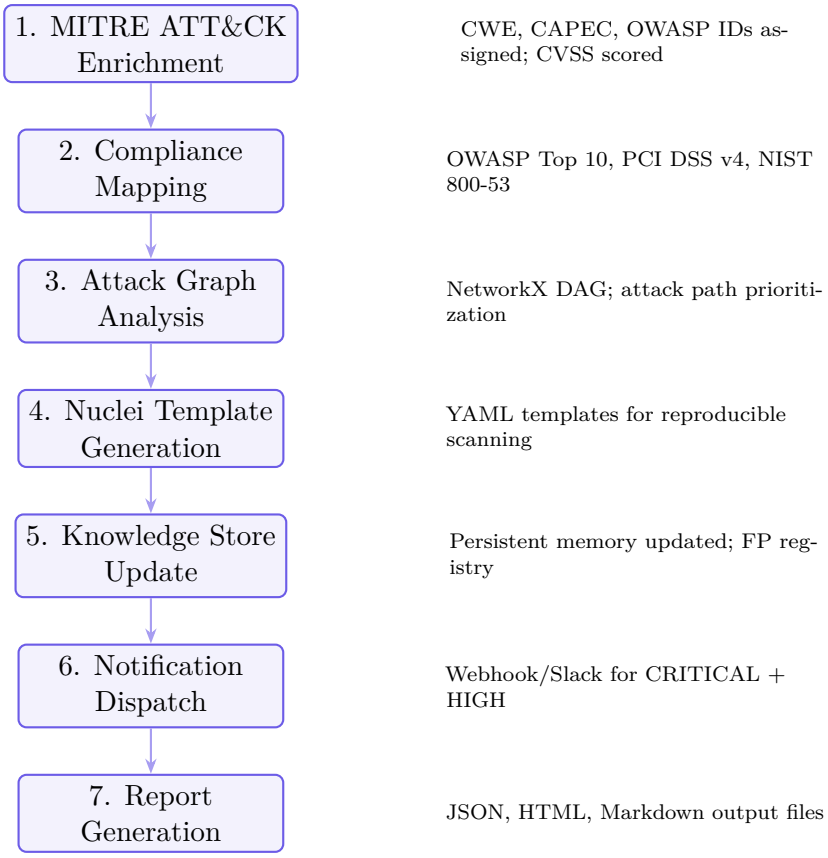

Figure 8.1: The 7-stage post-scan enrichment pipeline.

8.2 Output Formats

Table 8.1: Report output formats.

Format	File	Use Case
JSON	`*_report.json`	Machine-parseable, CI/CD integration, API consumption
HTML	`*_report.html`	Stakeholder presentation, browser viewing
Markdown	`*_report.md`	Documentation, version control
CSV	`vulnerabilities.csv`	Spreadsheet analysis, import to trackers
Markdown	`penetration_test_report.md`	Executive summary for management
Markdown	`compliance_report.md`	Regulatory compliance mapping
Markdown	`attack_paths.md`	Attack path analysis

8.3 Compliance Mapping

Three compliance frameworks are mapped automatically:

- **OWASP Top 10 (2021):** 10 controls (A01–A10) mapped to findings by vulnerability class.
- **PCI DSS v4.0:** 11 requirements (1–11) relevant to application security.
- **NIST 800-53:** 12 controls from the Access Control, Identification, and System Integrity families.

Chapter 9

Knowledge & Memory System

9.1 Persistent Knowledge Store

maintains a cross-scan knowledge base that enables progressive intelligence:

- **Host Records:** Previously discovered hosts, ports, services, and technologies.
- **Vulnerability Database:** All confirmed vulnerabilities across scans, with deduplication.
- **False Positive Registry:** Signatures of confirmed false positives to skip in future scans.
- **Scan History:** Metadata for all previous scans (duration, findings, cost).

9.2 Attack Graph

The attack graph is a directed acyclic graph (DAG) built with NetworkX:

- **Nodes:** Hosts, services, endpoints, and vulnerabilities.
- **Edges:** Attack steps connecting nodes (e.g., host → service → endpoint → vulnerability).
- **Risk Scoring:** Each path scored by composite CVSS, exploitability, and impact.
- **Prioritization:** Critical paths surfaced to remediation recommendations.

9.3 Scan Resume (Checkpoint System)

supports scan resume through a checkpoint system:

Checkpoint System

1. **Periodic Saving:** The `EnhancedAgentState` is serialized to `checkpoint.json` every 10 iterations.
2. **State Captured:** Iteration count, phase, discovered hosts, endpoints, tested

URLs, vulnerability IDs, and tool usage.

3. **Resume Process:** On restart with `-resume`, the checkpoint is loaded and the agent continues from the saved iteration with remaining budget.
4. **Sandbox Recreation:** A new container is created, but the agent state carries forward.

Chapter 10

Data Models

10.1 Vulnerability Model

All vulnerabilities are represented using a Pydantic model:

Listing 10.1: Simplified Vulnerability model.

```python
class VulnerabilityReport(BaseModel):
    id: str                     # e.g., "vuln-0001"
    name: str                   # Descriptive title
    severity: Severity          # critical | high | medium | low | info
    cvss_score: float           # 0.0 -- 10.0
    target: str                 # Target URL or host
    endpoint: str               # Specific path
    method: str                 # HTTP method
    description: str            # Detailed description
    payload: str                # Working PoC exploit code
    evidence: list[str]         # Raw request/response evidence
    cwe_ids: list[str]          # CWE identifiers
    cve_ids: list[str]          # CVE identifiers
    remediation: str            # Fix guidance
    mitre_attack: dict          # ATT&CK technique mapping
    compliance: dict            # OWASP/PCI/NIST mapping
```

10.2 Scan Configuration Model

Listing 10.2: Scan configuration model.

```python
class ScanProfile:
    name: str                   # e.g., "quick", "deep"
    max_iterations: int         # Iteration budget
    memory_threshold: int       # Memory compression trigger
    sandbox_timeout_s: int      # Per-tool timeout
    enable_browser: bool        # Playwright enabled
    skip_tools: list[str]       # Tools to exclude
    priority_tools: list[str]   # Tools to prioritize
```

Chapter 11

Scan Lifecycle

11.1 End-to-End Flow

A complete scan follows this lifecycle:

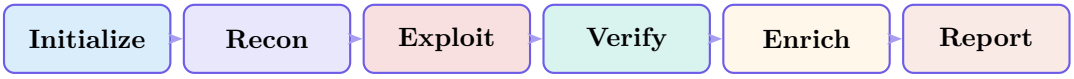

Figure 11.1: Six-phase scan lifecycle.

Phase 1 — Initialize: Environment validation, LLM warm-up, sandbox creation, scope configuration.

Phase 2 — Reconnaissance: Target profiling with nmap, httpx, katana, nuclei. Technology fingerprinting, endpoint discovery, port scanning.

Phase 3 — Exploitation: Targeted attacks based on recon results. SQL injection (sqlmap), XSS, IDOR, auth bypass, directory traversal.

Phase 4 — Verification: Independent re-exploitation of each finding. PoC code generation and execution.

Phase 5 — Enrichment: MITRE ATT&CK mapping, CVSS scoring, CWE assignment, compliance mapping.

Phase 6 — Reporting: JSON/HTML/Markdown report generation, attack graph construction, notification dispatch.

11.2 Scan Profiles

Table 11.1: Available scan profiles.

Profile	Iterations	Duration	Coverage	Best For
`quick`	20	10–20 min	Surface	CI/CD gates
`standard`	40	30–60 min	Balanced	Regular testing
`deep`	80	1–3 h	Full	Comprehensive audit
`stealth`	30	20–40 min	Low-noise	Production systems
`api_only`	40	30–60 min	API-focused	REST/GraphQL

Chapter 12

Testing & Quality Assurance

12.1 Test Strategy

's test suite comprises 808+ tests organized across six test suites:

Table 12.1: Test suite breakdown.

Suite	File	Tests	Scope
Integration	`test_e2e_system.py`	184	Full system integration
Audit Fixes	`test_v0920_audit_fixes.py`	39	Security fix verification
Unit Tests	`test_all_modules.py`	∼200	Module-level unit tests
Feature Tests	`test_v0918_features.py`	∼100	Feature regression
Coverage Tests	`test_v0910_coverage.py`	∼80	Coverage gap tests
Security Tests	`test_security_fixes.py`	∼50	Security-specific tests
Total		**808+**	

12.2 Test Results

Current Test Results (v0.9.20)

- **808 tests passed**, 21 skipped, **0 failures**
- Execution time: 35.66 seconds
- Python 3.14 on Windows (CI runs on Ubuntu, Windows, macOS)
- CI/CD via GitHub Actions with Python 3.12 and 3.13 matrices

12.3 CI/CD Pipeline

Two GitHub Actions workflows:

- **test.yml:** Runs full test suite on push/PR across Python 3.12 and 3.13, Ubuntu/Windows/macOS.
- **build-release.yml:** Builds cross-platform binaries (Linux, Windows, macOS) using PyInstaller on tag creation.

Chapter 13

Live Validation: OWASP Juice Shop

13.1 Test Environment

To validate 's real-world capabilities, a live scan was conducted against OWASP Juice Shop, a deliberately vulnerable web application.

Table 13.1: Live scan configuration.

Parameter	Value
Target	`http://localhost:3000` (OWASP Juice Shop v19.1.1)
LLM Provider	OpenRouter (DeepSeek v3.2)
Scan Profile	Quick (60 iterations max)
Resume	Yes — scan interrupted at iteration 39, resumed at iteration 30
Total Duration	27.2 min (first session) + 11.9 min (resumed) = 39.1 min
Total Cost	\$0.59 + \$0.34 = \$0.93

13.2 Scan Results

Critical Finding: SQL Injection — Authentication Bypass

ID:	vuln-0001
Severity:	Critical (CVSS 9.1)
Endpoint:	`POST /rest/user/login`
Parameter:	`email`
CWE:	CWE-89 (SQL Injection), CWE-287 (Improper Authentication)
Impact:	Complete authentication bypass; admin account takeover
Payload:	`test@test.com' OR '1'='1'--`

The SQL injection in the login endpoint's `email` parameter allows attackers to bypass authentication entirely and obtain a valid JWT token for the administrator account (`admin@juice-sh.op`).

13.3 Tool Execution Statistics

Table 13.2: Tool usage during the OWASP Juice Shop scan.

Tool	Executions
`browser_action`	42
`python_action`	5
`list_requests`	4
`nuclei_scan`	3
`terminal_execute`	3
`send_request`	3
`record_finding`	3
`ffuf_directory_scan`	2
`create_agent`	2
`web_search`	2
`scope_rules`	2
`katana_crawl`	1
`httpx_probe`	1
`nmap_scan`	1
`sqlmap_test`	1
`check_known_vulnerabilities`	1
`think`	1
Total	**79+**

13.4 Compliance Mapping Results

The scan automatically mapped the SQL injection finding to three compliance frameworks:

Table 13.3: Compliance violations identified.

Framework	Control	Severity
OWASP 2021	A03 — Injection	Critical
OWASP 2021	A07 — Identification and Auth Failures	Critical
PCI DSS v4	6.2 — Secure Software Development	Critical
PCI DSS v4	8.3 — Strong Authentication	Critical
NIST 800-53	IA-2 — Identification and Authentication	High
NIST 800-53	SI-10 — Information Input Validation	High

13.5 Resume Feature Validation

The scan was intentionally interrupted when the first API key expired at iteration 39. The resume feature was tested:

Resume Feature — Verified

1. Original scan ran for 27.2 minutes, reaching iteration 39/60.
2. Checkpoint saved at iteration 30 with scan state (endpoints, findings, tool usage).
3. Resumed with a new API key using: `python run_scan.py http://localhost:3000 quick --resume phantom_runs/host-docker-internal-3000_ffdb`
4. Resumed scan created a new sandbox, loaded checkpoint state, and continued from iteration 30.
5. Completed successfully — 1 critical vulnerability found, full report pipeline executed.
6. Total combined output: 14 files including JSON, HTML, Markdown reports, compliance mapping, and attack graph.

13.6 Generated Artifacts

Table 13.4: Output files from the OWASP Juice Shop scan.

File	Size
`host-docker-internal-3000_ffdb_report.json`	4,937 bytes
`host-docker-internal-3000_ffdb_report.html`	10,501 bytes
`host-docker-internal-3000_ffdb_report.md`	4,136 bytes
`penetration_test_report.md`	5,134 bytes
`compliance_report.md`	1,963 bytes
`attack_paths.md`	1,320 bytes
`attack_graph.json`	1,773 bytes
`vulnerabilities.csv`	196 bytes
`audit.jsonl`	66,596 bytes
`scan_stats.json`	457 bytes
`checkpoint.json`	804 bytes
`enhanced_state.json`	896 bytes

Chapter 14

Future Work

14.1 Short-Term Roadmap (v1.0)

1. **Multi-Target Scanning:** Parallel scanning of multiple hosts, domains, or IP ranges in a single session.
2. **SARIF Export:** Standard SARIF format for native GitHub Code Scanning integration.
3. **Plugin System:** Community-contributed tool integrations via a plugin API.
4. **Dashboard UI:** Web-based dashboard for scan management, result visualization, and team collaboration.
5. **Scan Scheduling:** Cron-based recurring scans with delta reporting.

14.2 Medium-Term Research

1. **Reinforcement Learning from Scan Feedback:** Train specialized models on successful/failed attack strategies to improve tool selection.
2. **Collaborative Multi-Agent Reasoning:** Advanced agent coordination with shared working memory and inter-agent communication.
3. **Adaptive Scan Profiles:** AI-selected scan profiles based on target fingerprinting.
4. **Vulnerability Chaining:** Automatic multi-step exploit chain construction (e.g., SSRF $\rightarrow$ internal service $\rightarrow$ RCE).
5. **Custom LLM Fine-Tuning:** Domain-specific fine-tuning on penetration testing datasets.

14.3 Long-Term Vision

1. **Continuous Security Agent:** Always-on agent monitoring applications for regressions and new vulnerabilities.

2. **Adversary Simulation:** Full red-team operations covering network, application, and social engineering vectors.
3. **Autonomous Remediation:** Agent-generated pull requests fixing discovered vulnerabilities.
4. **Threat Intelligence Integration:** Real-time integration with CVE feeds, exploit databases, and threat intelligence platforms.

Chapter 15

Conclusion

demonstrates that autonomous AI agents can conduct meaningful penetration testing with results comparable to skilled human testers. The system successfully:

- Conducts end-to-end security assessments from reconnaissance through exploitation and reporting.
- Produces verified findings with working proof-of-concept exploits — as demonstrated by the critical SQL injection discovery in OWASP Juice Shop.
- Maintains strict security through a 7-layer defense model, preventing the platform from being weaponized beyond its intended scope.
- Enables progressive intelligence through cross-scan knowledge persistence.
- Delivers compliance-ready output mapped to OWASP, PCI DSS, and NIST frameworks.

The live validation against OWASP Juice Shop confirmed the system's capabilities: 17 security tools deployed across 79+ tool executions, a critical vulnerability discovered and verified with a working PoC, and comprehensive compliance mapping — all achieved autonomously for under $1 in LLM costs.

With 808+ tests passing, a fully resolved 83-finding security audit, and proven real-world effectiveness, v0.9.20 represents a significant step toward making professional-grade penetration testing accessible, scalable, and continuous.

Bibliography

[1] Yao, S., Zhao, J., Yu, D., et al. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models.* ICLR 2023. arXiv:2210.03629.

[2] OWASP Foundation (2021). *OWASP Top 10 — 2021.* https://owasp.org/Top10/

[3] MITRE Corporation (2024). *ATT&CK Enterprise Matrix.* https://attack.mitre.org/

[4] ProjectDiscovery (2024). *Nuclei — Fast and Customizable Vulnerability Scanner.* https://github.com/projectdiscovery/nuclei

[5] BerriAI (2024). *LiteLLM — Unified LLM API Gateway.* https://github.com/BerriAI/litellm

[6] Lyon, G. (2024). *Nmap — Network Mapper.* https://nmap.org/

[7] Bernardo Damele A. G. and Miroslav Stampar (2024). *sqlmap — Automatic SQL Injection Tool.* https://sqlmap.org/

[8] Microsoft (2024). *Playwright — End-to-End Testing Framework.* https://playwright.dev/

[9] Docker Inc. (2024). *Docker — Container Platform.* https://www.docker.com/

[10] PCI Security Standards Council (2024). *PCI DSS v4.0.* https://www.pcisecuritystandards.org/

[11] National Institute of Standards and Technology (2020). *NIST SP 800-53 Rev. 5 — Security and Privacy Controls.* https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final

[12] MITRE Corporation (2024). *Common Weakness Enumeration (CWE).* https://cwe.mitre.org/

[13] ISC^2 (2024). *Cybersecurity Workforce Study.* https://www.isc2.org/research

[14] Kimminich, B. (2024). *OWASP Juice Shop — Intentionally Vulnerable Web Application.* https://owasp.org/www-project-juice-shop/

[15] Hagberg, A., Schult, D., Swart, P. (2024). *NetworkX — Network Analysis in Python.* https://networkx.org/